

Situation d'Apprentissage et d'Évaluation (SAÉ) **S2.04 : Exploitation d'une base de données**

Sujet : Analyse des stations-service et des prix des carburants
à partir de données ouvertes

PATHMARANJAN Kuruparan
KATEB Yacine
SERCEK Yusa

Groupe 4 : K2Y DATA

BUT Informatique Venus

SOMMAIRE

1. Organisation du projet

- 1.1. Constitution de l'équipe
- 1.2. Répartition des rôles et des tâches
- 1.3. Choix des outils
- 1.4. Planning prévisionnel
- 1.5. Diagramme de Gantt

2. Modélisation des données

- 2.1. Présentation rapide des données exploitées
- 2.2. Identification des principales entités
- 2.3. Relations entre les tables
- 2.4. Les clés primaires et les clés étrangères
- 2.5. MCD, MLD et schéma relationnel
- 2.6. Justification des principaux choix de modélisation

3. Base de données et requêtes SQL

- 3.1. Scripts SQL de création des tables
- 3.2. Clés primaires et les clés étrangères
- 3.3. Contraintes d'intégrité
- 3.4. Scripts d'importation ou d'alimentation des données
- 3.5. Explication du mécanisme d'historisation des prix
- 3.6. Requetes SQL d'analyse
- 3.7. Interprétation des premiers résultats obtenus

4. Analyse et visualisation

- 4.1. Visualisations produites
- 4.2. Synthèse des résultats obtenus
- 4.3. Conclusion sur les principaux enseignements du projet

1. Organisation du projet

Ce projet a pour objectif de concevoir et développer une base de données relationnelle permettant d'exploiter des données open data sur les stations-service et les prix des carburants. L'objectif final est d'analyser ces données afin de comparer les prix des carburants, suivre leur évolution dans le temps et identifier les stations-service les plus avantageuses selon différents critères.

1.1. Constitution de l'équipe

Cette équipe dont le nom est : **K2Y DATA** est constitué de 3 étudiants :

- PATHMARANJAN Kuruparan (chef d'équipe)
- KATEB Yacine
- SERCEK Yusa

Afin d'assurer une bonne coordination entre les membres de l'équipe, plusieurs outils de communication seront utilisés :

- WhatsApp : communication rapide en dehors des séances ;
- Google Drive : partage des documents et sauvegarde du travail.

L'objectif est de maintenir la communication pour la prise de décision, un suivi régulier et des rendus ponctuels.

1.2. Répartition des rôles et des tâches

Chaque membre de l'équipe doit maîtriser l'ensemble du projet. Cependant, une répartition des tâches a été mise en place afin de faciliter l'organisation et le démarrage du travail.

Membre	Rôle principal	Tâches principales
PATHMARANJAN Kuruparan	Modélisation des données	Analyse des données, création du MCD/MLD, schéma relationnel et mise en forme des rendus
KATEB Yacine	Base de données et SQL	Création de la base PostgreSQL/SQLite, scripts SQL, importation des données
SERCEK Yusa	Analyse et visualisation	Création des tableaux de bord Grafana, graphiques et analyses

Certains travaux seront réalisés collectivement afin de garantir la maîtrise du projet par tous les membres de l'équipe :

- Vérification des requêtes SQL ;
- Tests de la base de données ;
- Analyse des difficultés rencontrées ;
- Compréhension globale du projet ;
- Préparation des différents rendus.

1.3. Choix des outils

Plusieurs outils seront utilisés durant la SAÉ afin de faciliter le développement du projet et le travail collaboratif :

- WhatsApp : communication rapide en dehors des séances ;
- Google Drive : partage des documents et sauvegarde du travail ;
- PostgreSQL ou SQLite : gestion de la base de données ;
- Python : traitement et importation des données ;
- Grafana : création des visualisations

1.4. Planning prévisionnel

Le projet sera organisé sur plusieurs semaines conformément aux échéances des différents livrables attendus.

11 mai → 18 mai : Organisation de l'équipe, lecture du sujet, choix des outils, réalisation du Livrable 1

18 mai → 24 mai : Analyse des données, création du modèle de données et du schéma relationnel

25 mai → 31 mai : Création de la base de données, importation des données et requêtes SQL

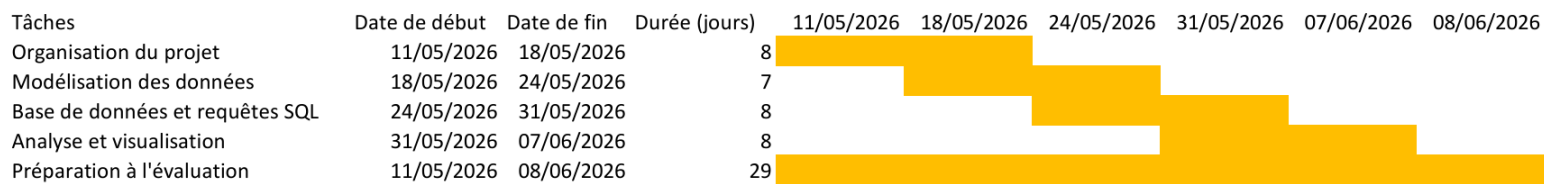
1 juin → 7 juin : Création des visualisations, tableau de bord Grafana et finalisation du projet

Tout au long de l'avancement du projet : préparation à l'évaluation

Objectifs intermédiaires

- Finir l'organisation du projet avant le 17 mai ;
- Réaliser un modèle de données cohérent avant le 24 mai ;
- Avoir une base de données fonctionnelle avec les requêtes SQL avant le 31 mai ;
- Réaliser les visualisations et terminer les analyses avant le 7 juin.

1.5. Diagramme de Gantt



Légende :

Durée des tâches



Description du diagramme

Ce diagramme de Gantt présente l'organisation prévisionnelle du projet sur l'ensemble de la SAÉ.

Chaque phase correspond à un livrable du projet :

- Organisation du projet ;
- Modélisation des données ;
- Développement de la base SQL ;
- Création des visualisations et finalisation.

Cette planification permettra de suivre l'avancement du projet et d'anticiper les différentes étapes du développement.

2. Modélisation des données

2.1. Présentation rapide des données exploitées

Pour la réalisation de cette base de données, les données exploitées viennent d'une source open data et contiennent des informations variées :

- les stations-service
- les carburants proposés
- les prix associés à chaque carburant
- les dates de mise à jour des prix
- les éventuelles ruptures
- les horaires d'ouverture
- les services disponibles

L'objectif est de regrouper ces informations dans une base de données relationnelle afin de faciliter leur traitement et leur analyse.

2.2. Identification des principales entités

Afin de bien modéliser notre base de données, nous avons identifié plusieurs entités principales :

- STATION : Cette entité représente chaque point de vente. Elle permet de stocker toutes les informations physiques de la station, comme son adresse et ses coordonnées géographiques.
- CARBURANT : Cette entité enregistre les différents types uniques de carburants disponibles . Cela évite de répéter à chaque fois le nom des produits dans la base de données.
- PRIX : Cette entité modélise l'évolution des tarifs au fil du temps. Le prix d'un carburant change souvent donc à chaque enregistrement dans cette entité correspond à une augmentation de prix associé à une certaine date défini .
- SERVICE : Cette entité liste de façon unique l'ensemble des services présents dans la station.
- RUPTURE : Cette entité permet de suivre toutes les fois où les carburants ne sont pas disponibles.

2.3. Relations entre les tables

Pour lier nos différentes entités et assurer la cohérence de la base de données, nous avons défini les relations suivantes :

1. Relation entre STATION et PRIX

Description : Une station-service peut enregistrer de nombreux prix. À l'inverse, un enregistrement d'historique des prix ne peut correspondre qu'à une seule station bien précise.

Cardinalités : Une station possède de 1 à plusieurs lignes de prix (1,n), et un prix est lié à une et une seule station (1,1).

2. Relation entre CARBURANT et PRIX

Description : Un type de carburant est concerné par plusieurs relevés dans différentes stations et différentes dates. Chaque ligne de prix enregistrée ne concerne qu'un seul type de carburant.

Cardinalités : Un carburant est associé à 1 ou plusieurs lignes de prix (1,n), et un prix concerne exactement un seul carburant (1,1).

3. Relation entre STATION et SERVICE

Description : Une station-service peut proposer plusieurs services annexes . Et un service peut être disponible dans un grand nombre de stations différentes.

Modélisation : Cette association maillée correspond à la table intermédiaire PROPOSE dans un schéma relationnel. Une ligne dans cette table associe une station à un service.

Cardinalités : Une station propose de 1 à plusieurs services (1,n), et un service est proposé par 1 à plusieurs stations (1,n).

4. Relation entre STATION et RUPTURE

Description : Une station-service peut subir plusieurs coupures de stock depuis sa création. Chaque signalement de rupture est lié à un unique point de vente concerné.

Cardinalités : Une station subit de 1 à plusieurs ruptures (1,n), et une rupture donnée concerne obligatoirement une seule station (1,1).

5. Relation entre CARBURANT et RUPTURE

Description : Un type de carburant spécifique peut être en rupture à des périodes ou dans des lieux différents. Un événement de rupture de stock ne concerne qu'un seul carburant à la fois.

Cardinalités : Un carburant peut subir de 1 à plusieurs ruptures (1,n), et un événement de rupture concerne un seul carburant (1,1).

2.4. Les clés primaires et les clés étrangères

Pour garantir l'unicité des enregistrements et matérialiser les liens de dépendance entre nos tables, nous avons défini l'ensemble des clés primaires et étrangères comme suit :

1. Table STATION

Clé Primaire : id_station

Type : VARCHAR

Justification : Cet identifiant est directement fourni par le jeu de données Open Data et est unique pour chaque point de vente.

2. Table CARBURANT

Clé Primaire : id_carburant

Type : VARCHAR.

3. Table PRIX

Clé Primaire : id_prix

Type : INTEGER avec GENERATED ALWAYS AS IDENTITY.

Clés Étrangères :

- id_station en référence à STATION(id_station) pour identifier précisément le point de vente concerné par ce tarif.
- id_carburant en référence à CARBURANT(id_carburant) pour spécifier à quel produit s'applique ce prix au litre.

4. Table SERVICE

Clé Primaire : id_service

Type : INTEGER avec auto-incrémentation

5. Table PROPOSE

Clé Primaire : Clé composée du couple (id_station, id_service).

Clés Étrangères :

- id_station en référence à STATION(id_station) pour désigner la station qui propose le service.
- id_service en référence à SERVICE(id_service) pour définir le service proposé.

6. Table RUPTURE

Clé Primaire : id_rupture

Type : INTEGER avec auto-incrémentation

Justification : Clé technique permettant de différencier chaque signalement ou événement d'indisponibilité de stock.

Clés Étrangères :

- id_station en référence à STATION(id_station) pour savoir où la coupure de stock a eu lieu.
- id_carburant en référence à CARBURANT(id_carburant) pour identifier quel carburant est en rupture de stock.

2.5. MCD et schéma relationnel

Schéma relationnel :

STATION (id_station, nom, adresse, code_postal, ville, latitude, longitude, horaires)

CARBURANT (id_carburant, nom_carburant)

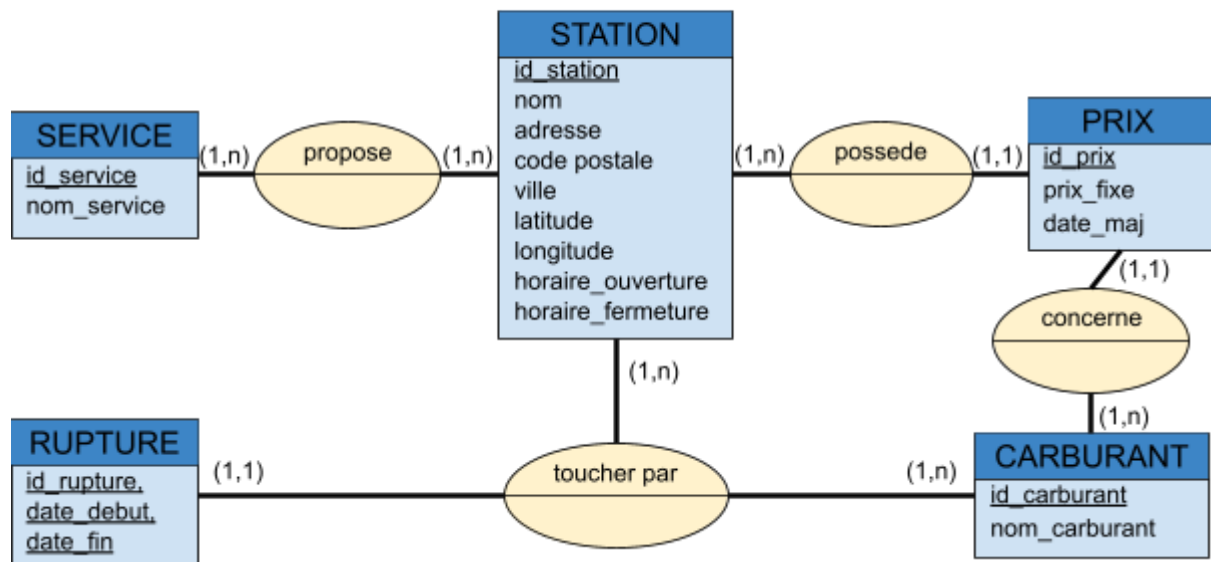
PRIX (id_prix, prix_fixe, date_maj, #id_station, #id_carburant)

SERVICE (id_service, nom_service)

RUPTURE (id_rupture, date_debut, date_fin, #id_station, #id_carburant)

PROPOSE (#id_station, #id_service)

MODÈLE CONCEPTUEL DE DONNÉES (MCD)



2.6. Justification des principaux choix de modélisation.

Cette modélisation a été conçue afin d'obtenir une base de données claire, cohérente et facile à exploiter. Les informations principales ont été séparées en plusieurs tables afin d'éviter les répétitions inutiles et de mieux organiser les données.

La table **STATION** regroupe les informations générales des stations-service, comme le nom, l'adresse, la ville, les coordonnées géographiques et l'horaire. La table **CARBURANT** permet de stocker une seule fois chaque type de carburant, ce qui évite de répéter les mêmes noms dans plusieurs tables.

La table **PRIX** a été créée séparément afin de conserver l'historique des prix dans le temps. Cela permet de suivre l'évolution des tarifs et de comparer les prix entre plusieurs stations. Les tables **RUPTURE** et **SERVICE** permettent quant à elles de gérer séparément les ruptures de carburant et les services proposés par les stations.

Enfin, la table **PROPOSE** est nécessaire car une station peut proposer plusieurs services, et un même service peut être présent dans plusieurs stations. Cette modélisation permet donc de respecter les relations entre les données et de faciliter les futures requêtes SQL du projet.

3. Base de données et requêtes SQL

Cette partie présente la mise en place de la base de données relationnelle. L'objectif est de créer les tables nécessaires, d'importer les données issues du fichier XML, puis d'exploiter la base à l'aide de requêtes SQL d'analyse.

3.1. Scripts SQL de création des tables

La base de données est composée de six tables principales : STATION, CARBURANT, SERVICE, PRIX, RUPTURE et PROPOSE.

```
CREATE TABLE STATION (  
    id_station INT PRIMARY KEY,  
    adresse VARCHAR(255),  
    cp VARCHAR(5),  
    ville VARCHAR(100),  
    latitude FLOAT,  
    longitude FLOAT,  
    horaires TEXT  
);
```

```
CREATE TABLE CARBURANT (  
    id_carburant INT PRIMARY KEY,  
    nom_carburant VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE SERVICE (  
    id_service INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    nom_service VARCHAR(255) NOT NULL UNIQUE  
);
```

```
CREATE TABLE PRIX (  
    id_prix INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    prix_fixe DECIMAL(5, 3) NOT NULL,  
    date_maj TIMESTAMP,  
    id_station INT REFERENCES STATION(id_station),  
    id_carburant INT REFERENCES CARBURANT(id_carburant)  
);
```

```
CREATE TABLE RUPTURE (  
    id_rupture INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    date_debut TIMESTAMP,  
    date_fin TIMESTAMP,  
    type_rupture VARCHAR(50),  
    id_station INT REFERENCES STATION(id_station),  
    id_carburant INT REFERENCES CARBURANT(id_carburant)  
);
```

```
CREATE TABLE PROPOSE (  
    id_station INT REFERENCES STATION(id_station),  
    id_service INT REFERENCES SERVICE(id_service),  
    PRIMARY KEY (id_station, id_service)  
);
```

3.2. Clés primaires et les clés étrangères

1) Table STATION :

- Clé primaire : id_station

2) Table CARBURANT :

- Clé primaire : id_carburant

3) Table SERVICE :

- Clé primaire : id_service

4) Table PRIX :

- Clé primaire : id_prix
- Clés étrangères : id_station / id_carburant

5) Table RUPTURE :

- Clé primaire : id_rupture
- Clés étrangères : id_station / id_carburant

6) Table PROPOSE :

- Clé primaire : (id_station, id_service)
- Clés étrangères : id_station / id_service

3.3. Contraintes d'intégrité

Tout d'abord, les différentes clés primaires assurent l'unicité de chaque ligne. Les clés étrangères, elles, montrent le lien entre les différentes tables. Par exemple, un prix nécessite d'être lié à une station et à un carburant existant. De plus, la contrainte NOT NULL empêche certaines valeurs d'être vides. Cela permet de rester cohérent avec les attributs. En effet, un carburant est obligé de posséder un nom.

La contrainte UNIQUE permet d'éviter les redondances dans les tables. En effet, quand on applique cette contrainte sur nom_service, cela évite d'avoir plusieurs fois le même service dans la table SERVICE.

3.4. Scripts d'importation ou d'alimentation des données

Avant l'importation dans la base, un script Python extrait les données du fichier XML PrixCarburants_annuel_2025.xml.

Ce script génère plusieurs fichiers CSV : carburants, services, ruptures et horaires.

Ensuite, le script SQL d'importation crée des tables temporaires afin de charger les données CSV.

Script Python :

```
import xml.etree.ElementTree as ET
import os
```

```
fichier_xml = 'PrixCarburants_annuel_2025.xml'
```

```
if not os.path.exists(fichier_xml):
    print("Erreur : fichier XML introuvable.")
    exit()
```

```
print("Extraction globale en cours... (Lecture unique du fichier XML)")
```

```
# 1. Ouverture de tous les fichiers CSV en écriture
```

```
f_carb = open('import_carburants.csv', "w", encoding="utf-8")
```

```
f_carb.write("id_station,cp,ville,adresse,id_carburant,nom_carburant,prix,date_maj\n")
```

```
f_serv = open('import_services.csv', "w", encoding="utf-8")
```

```
f_serv.write("id_station,nom_service\n")
```

```
f_rup = open('import_ruptures.csv', "w", encoding="utf-8")
```

```
f_rup.write("id_station,id_carburant,date_debut,date_fin,type_rupture\n")
```

```
f_hor = open('import_horaires.csv', "w", encoding="utf-8")
```

```
f_hor.write("id_station,horaires\n")
```

```
# 2. Analyse de l'arbre XML
```

```
tree = ET.parse(fichier_xml)
```

```
root = tree.getroot()
```

```
# 3. Boucle unique sur chaque Point de Vente (pdv)
```

```
for pdv in root.findall('pdv'):
```

```
    id_station = pdv.get('id')
```

```
    cp = pdv.get('cp')
```

```
# -- Adresse et Ville --
```

```
elem_adresse = pdv.find('adresse')
```

```
if elem_adresse is not None and elem_adresse.text is not None:
```

```
    adresse = elem_adresse.text.replace(',', ' ')
```

```
else:
```

```
    adresse = ""
```

```
elem_ville = pdv.find('ville')
```

```
if elem_ville is not None and elem_ville.text is not None:
```

```
    ville = elem_ville.text.replace(',', ' ')
```

```
else:
```

```
    ville = ""
```

```
# -- Carburants et Prix --
```

```
for prix in pdv.findall('prix'):
```

```
    id_carburant = prix.get('id')
```

```
    nom_carburant = prix.get('nom')
```

```
    valeur = prix.get('valeur')
```

```
    date_maj = prix.get('maj')
```

```
if id_carburant is not None and valeur is not None:
```

```
f_carb.write(f'{id_station},{cp},{ville},{adresse},{id_carburant},{nom_carburant},{valeur},{date_maj}\n')
```

```
# -- Services --
services = pdv.find('services')
if services is not None:
    for service in services.findall('service'):
        if service.text:
            nom_service = service.text.replace(',', ' ').strip()
            f_serv.write(f'{id_station},{nom_service}\n')

# -- Ruptures --
for rupture in pdv.findall('rupture'):
    id_carburant_rup = rupture.get('id')
    type_rupture = rupture.get('nom')
    date_debut = rupture.get('debut')
    date_fin = rupture.get('fin')

    if date_fin is None:
        date_fin = ""

    if id_carburant_rup is not None:

f_rup.write(f'{id_station},{id_carburant_rup},{date_debut},{date_fin},{type_rupture}
\n")

# -- Horaires --
horaires_elem = pdv.find('horaires')
texte_horaire = "Non renseigné"
if horaires_elem is not None:
    if horaires_elem.get('automate-24-24') == '1':
        texte_horaire = "Automate 24h/24"
    else:
        texte_horaire = "Horaires standards"
f_hor.write(f'{id_station},{texte_horaire}\n')

# 4. Fermeture de tous les fichiers
f_carb.close()
```

```
f_serv.close()
f_rup.close()
f_hor.close()
```

```
print("Extraction totale terminée ! Tous les fichiers CSV sont prêts.")
```

-- 1. Création des tables temporaires

```
CREATE TEMP TABLE tmp_carburants (
    id_station INT, cp VARCHAR(5), ville VARCHAR(100), adresse
    VARCHAR(255),
    id_carburant INT, nom_carburant VARCHAR(50), prix NUMERIC(5,3),
    date_maj TIMESTAMP
);
```

```
CREATE TEMP TABLE tmp_services (
    id_station INT, nom_service VARCHAR(255)
);
```

```
CREATE TEMP TABLE tmp_ruptures (
    id_station INT, id_carburant INT, date_debut TIMESTAMP, date_fin
    TIMESTAMP, type_rupture VARCHAR(50)
);
```

```
CREATE TEMP TABLE tmp_horaires (
    id_station INT, horaires TEXT
);
```

-- 2. Importation des fichiers CSV

```
\copy tmp_carburants FROM 'import_carburants.csv' WITH (FORMAT csv,
HEADER true, DELIMITER ',');
\copy tmp_services FROM 'import_services.csv' WITH (FORMAT csv,
HEADER true, DELIMITER ',');
\copy tmp_ruptures FROM 'import_ruptures.csv' WITH (FORMAT csv,
HEADER true, DELIMITER ',', NULL '');
\copy tmp_horaires FROM 'import_horaires.csv' WITH (FORMAT csv,
HEADER true, DELIMITER ',');
```


-- 3. Remplissage des tables finales

-- Ajout des stations

```
INSERT INTO STATION (id_station, cp, ville, adresse)
SELECT id_station, MAX(cp), MAX(ville), MAX(adresse)
FROM tmp_carburants
WHERE id_station NOT IN (SELECT id_station FROM STATION)
GROUP BY id_station;
```

-- Mise à jour des horaires

```
UPDATE STATION
SET horaires = tmp_horaires.horaires
FROM tmp_horaires
WHERE STATION.id_station = tmp_horaires.id_station;
```

-- Ajout des carburants

```
INSERT INTO CARBURANT (id_carburant, nom_carburant)
SELECT id_carburant, MAX(nom_carburant)
FROM tmp_carburants
WHERE id_carburant NOT IN (SELECT id_carburant FROM CARBURANT)
GROUP BY id_carburant;
```

-- Ajout des services

```
INSERT INTO SERVICE (nom_service)
SELECT DISTINCT nom_service
FROM tmp_services
WHERE nom_service IS NOT NULL
AND nom_service NOT IN (SELECT nom_service FROM SERVICE);
```

-- Ajout des liaisons entre stations et services

```
INSERT INTO PROPOSE (id_station, id_service)
SELECT DISTINCT t.id_station, s.id_service
FROM tmp_services t
JOIN SERVICE s ON t.nom_service = s.nom_service
WHERE NOT EXISTS (
    SELECT 1 FROM PROPOSE p
    WHERE p.id_station = t.id_station AND p.id_service = s.id_service
);
```

-- Ajout des prix

```
INSERT INTO PRIX (id_station, id_carburant, prix_fixe, date_maj)
SELECT id_station, id_carburant, prix, date_maj FROM tmp_carburants;
```

-- Ajout des ruptures

```
INSERT INTO RUPTURE (id_station, id_carburant, date_debut, date_fin,
type_rupture)
SELECT id_station, id_carburant, date_debut, date_fin, type_rupture FROM
tmp_ruptures;
```

-- 4. Suppression des tables temporaires

```
DROP TABLE tmp_carburants, tmp_services, tmp_ruptures, tmp_horaires;
```

3.5. Explication du mécanisme d'historisation des prix

L'historisation est rendue possible par la séparation des données entre la table STATION, qui contient les informations fixes, et la table PRIX, qui gère les données changeantes. Nous ne stockons pas le prix directement dans la table STATION, car cela écraserait l'ancienne valeur à chaque mise à jour. Nous utilisons une table PRIX dédiée où chaque ligne est liée à une station par sa clé étrangère, l'id_station, et au carburant par son id_carburant.

Chaque enregistrement possède un TIMESTAMP, un format de données qui enregistre la date et l'heure précise de la mise à jour. Grâce à ce mécanisme, chaque modification génère une nouvelle ligne dans la table PRIX. L'ancienne donnée est ainsi conservée, ce qui nous permet de consulter l'historique complet des variations d'un prix.

Par ailleurs, grâce à l'heure et à la date, nous pouvons extraire l'évolution des prix sur une période donnée pour identifier des tendances. Enfin, la table STATION reste légère tandis que la table PRIX sert de journal de bord. Cela facilite les requêtes d'analyse sans risquer de perdre les informations sur la description des stations.

3.6. Requêtes SQL d'analyse

-- Pour que tout s'affiche directement dans le terminal

```
\pset pager off
```

-- 1. Afficher les stations et les carburants disponibles

```
SELECT s.id_station, s.ville, c.nom_carburant
FROM STATION s
JOIN PRIX p ON s.id_station = p.id_station
JOIN CARBURANT c ON p.id_carburant = c.id_carburant
GROUP BY s.id_station, s.ville, c.nom_carburant
ORDER BY s.ville
LIMIT 10;
```

-- 2. Comparer les prix du Gazole tel que le E7 entre plusieurs stations

```
SELECT s.ville, s.adresse, p.prix_fixe, p.date_maj
FROM STATION s
JOIN PRIX p ON s.id_station = p.id_station
JOIN CARBURANT c ON p.id_carburant = c.id_carburant
WHERE c.nom_carburant ILIKE '%Gazole%'
ORDER BY p.prix_fixe ASC
LIMIT 10;
```

-- 3. Identifier les stations les moins chères pour le SP95-E10

```
SELECT s.id_station, s.ville, s.adresse, p.prix_fixe
FROM STATION s
JOIN PRIX p ON s.id_station = p.id_station
JOIN CARBURANT c ON p.id_carburant = c.id_carburant
WHERE c.nom_carburant ILIKE '%E10%'
AND p.prix_fixe = (
    SELECT MIN(p2.prix_fixe)
    FROM PRIX p2
    JOIN CARBURANT c2 ON p2.id_carburant = c2.id_carburant
    WHERE c2.nom_carburant ILIKE '%E10%'
)
LIMIT 10;
```

-- 4. Analyser le nombre de ruptures par type de carburant

```
SELECT c.nom_carburant, COUNT(r.id_rupture) as nombre_de_ruptures
FROM CARBURANT c
LEFT JOIN RUPTURE r ON c.id_carburant = r.id_carburant
GROUP BY c.nom_carburant
ORDER BY nombre_de_ruptures DESC
LIMIT 10;
```

-- 5. Exploiter les horaires d'ouverture comme la répartition des types d'horaires

```
SELECT horaires, COUNT(id_station) as nombre_de_stations
FROM STATION
GROUP BY horaires
ORDER BY nombre_de_stations DESC
LIMIT 10;
```

-- 6. Calculer des indicateurs simples tels que le prix min max et moyen par carburant

```
SELECT c.nom_carburant,
       ROUND(AVG(p.prix_fixe), 3) as prix_moyen,
       MIN(p.prix_fixe) as prix_minimum,
       MAX(p.prix_fixe) as prix_maximum,
       COUNT(DISTINCT p.id_station) as nb_stations_proposant
FROM CARBURANT c
LEFT JOIN PRIX p ON c.id_carburant = p.id_carburant
GROUP BY c.nom_carburant
ORDER BY prix_moyen ASC
LIMIT 10;
```

3.7. Interprétation des premiers résultats obtenus

Stations et carburants : L'affichage des 10 premières lignes montre une bonne diversité avec des stations comme celle d'Abbeville qui proposent une large gamme de carburants incluant le Gazole, l'E10 ainsi que le SP95 et le SP98, ce qui confirme que nos jointures entre tables fonctionnent correctement.

Comparaison tarifaire : Les 10 résultats affichés pour les prix les plus bas révèlent des tarifs attractifs autour de 1,21 euro répartis sur tout le territoire, ce qui prouve que notre base est un outil efficace pour identifier les stations les plus compétitives.

Station la moins chère : Le résultat unique pour l'E10 à 1,261 euro montre que nos requêtes SQL sont capables d'isoler précisément une information pertinente concernant le prix minimum absolu au milieu d'un grand volume de données.

Gestion des ruptures : Le classement des 6 carburants montre que l'E10 et le SP98 sont les plus touchés par les ruptures avec plus de 10 000 cas

chacun, ce qui met en évidence des tensions réelles sur ces approvisionnements.

Modernisation des services : La répartition montre que les automates 24h/24 avec 5 286 stations sont désormais plus fréquents que les stations à horaires classiques qui en comptent 4 440, ce qui illustre une tendance forte vers l'automatisation.

Cohérence statistique : Les prix moyens calculés par carburant allant de 0,764 euro pour l'E85 jusqu'à 1,817 euro pour le SP98 sont tout à fait cohérents avec les tarifs du marché actuel, ce qui valide la fiabilité de notre travail d'importation.

Résultats obtenus :

id_station	ville	nom_carburant		
80100011	abberville	E10		
80100011	abberville	SP98		
80100011	abberville	Gazole		
80100012	Abbeville	Gazole		
80100007	Abbeville	E10		
80100012	Abbeville	E85		
80100007	Abbeville	SP98		
80100007	Abbeville	Gazole		
80100012	Abbeville	E10		
80100012	Abbeville	SP95		
(10 rows)				
ville	adresse	prix_fixe	date_maj	
FOIX	Route d'Espagne	1.213	2025-04-18 07:38:34	
SAINT-NAZAIRE	54 Rue Léon Bourgeois	1.214	2025-12-22 05:12:56	
Vernou-sur-Brenne	25 Rue du Professeur Robert Debré	1.215	2025-06-06 07:22:28	
BOISSISE-LA-BERTRAND	Avenue Jean Monnet	1.221	2025-04-18 09:19:20	
Auxonne	AVENUE GENERAL DE GAULLE	1.226	2025-05-05 12:10:00	
Oyonnax	74 Rue Jules Michelet	1.231	2025-12-16 08:03:48	
Avignon	10 Avenue Eisenhower	1.232	2025-12-12 08:28:06	
Erquinghem-Lys	Rue d'Armentières	1.234	2025-04-19 10:16:18	
BOISSISE-LA-BERTRAND	Avenue Jean Monnet	1.239	2025-04-18 09:12:46	
AVORD	2 rue Agnès Sorel	1.241	2025-04-18 10:07:50	
(10 rows)				
id_station	ville	adresse	prix_fixe	
13630004	Eyragues	149 LES ALLEES	1.261	
(1 row)				
nom_carburant	nombre_de_ruptures			
E10	11132			
SP98	10014			
GPLc	9543			
SP95	8198			
E85	6903			
Gazole	4752			
(6 rows)				
horaires	nombre_de_stations			
Automate 24h/24	5286			
Horaires standards	4440			
(2 rows)				
nom_carburant	prix_moyen	prix_minimum	prix_maximum	nb_stations_proposant
E85	0.764	0.385	1.999	3983
GPLc	0.992	0.496	1.989	1536
Gazole	1.648	1.213	3.000	9712
E10	1.713	1.261	2.664	7702
SP95	1.727	1.326	2.889	3610
SP98	1.817	1.296	2.997	8236
(6 rows)				

4. Analyse et visualisation

4.1. Visualisations produites

Analyse des ruptures de carburant :



Ce graphique représente le **nombre de ruptures de stock par type de carburant**.

On voit que :

- **E10** est le carburant qui connaît le plus de ruptures, avec environ **11 000 ruptures**.
- **SP98** et **GPLc** suivent avec respectivement environ **10 000** et **9 500 ruptures**.
- **SP95** présente un nombre de ruptures légèrement inférieur, autour de **8 000**.
- **E85** enregistre environ **6 800 ruptures**.
- **Gazole** est le carburant le moins touché, avec environ **4 700 ruptures**.

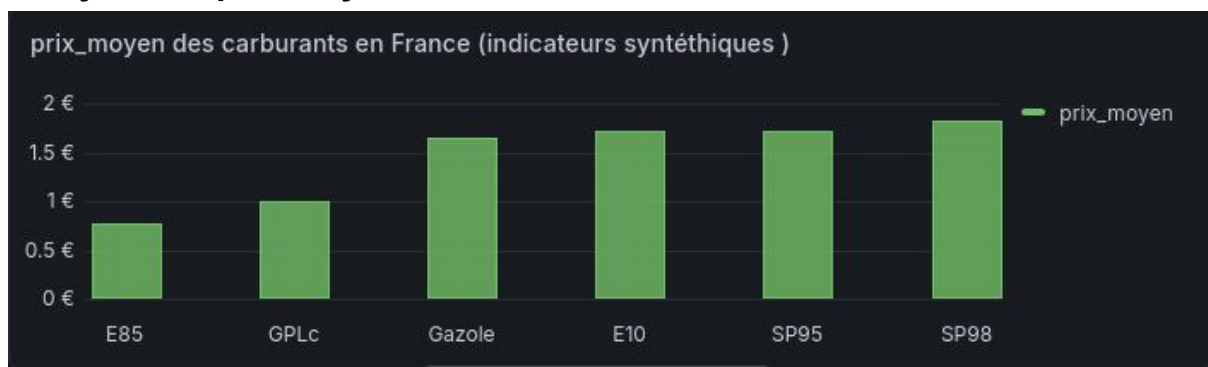
Les **carburants essence** sont davantage concernés par les ruptures : les carburants **E10**, **SP98**, **GPLc** et **SP95** concentrent le plus grand nombre de ruptures. Cela peut refléter une forte demande ou une gestion des stocks plus complexe pour ces carburants.

Le **Gazole** semble mieux approvisionné. Malgré son importance dans le parc automobile, il présente le plus faible nombre de ruptures. Cela peut s'expliquer par une logistique d'approvisionnement plus robuste ou par des stocks plus importants.

L'**E85** occupe une position intermédiaire Son nombre de ruptures est inférieur à celui des autres essences mais supérieur à celui du Gazole. Son réseau de distribution étant plus limité, la demande et les stocks peuvent être mieux équilibrés.

L'analyse met en évidence que les ruptures de stock concernent principalement les carburants essence, en particulier l'E10, tandis que le Gazole apparaît comme le carburant le plus disponible sur la période étudiée. Ces résultats suggèrent que les stratégies d'approvisionnement et les niveaux de consommation varient selon le type de carburant.

Analyse des prix moyen des différents carburants en France :



Ce graphique présente le **prix moyen des différents carburants en France sur la période étudiée**.

On voit que le :

- **SP98** est le carburant le plus cher, avec un prix moyen d'environ **1,80 €/L.**
- **SP95** et **E10** suivent de près, avec un prix moyen proche de **1,70 €/L.**
- **Gazole** se situe légèrement en dessous, autour de **1,65 €/L.**
- **GPLc** est nettement moins cher, avec un prix moyen d'environ **1,00 €/L.**
- **E85** est le carburant le moins coûteux, avec un prix moyen d'environ **0,75 €/L.**

Les **carburants essence** traditionnels sont les plus coûteux. Le **SP98**, le **SP95** et l'**E10** affichent les prix moyens les plus élevés. Cette différence peut s'expliquer par leur composition, leur niveau de raffinage et la fiscalité appliquée. Le **Gazole** conserve un prix relativement compétitif. Bien qu'il reste proche des carburants essence, son prix moyen demeure légèrement inférieur. Cela contribue à maintenir son attractivité auprès de certains automobilistes. Les **carburants alternatifs** sont nettement moins chers L'**E85** et le **GPLc** bénéficient d'une fiscalité avantageuse et d'un coût de production plus faible. L'**E85** apparaît comme l'option la plus économique pour les véhicules compatibles.

Les résultats montrent une forte disparité des prix selon le type de carburant. Le SP98 est le plus onéreux, tandis que l'E85 est de loin le plus économique, avec un prix moyen inférieur de plus d'un euro par litre par rapport aux essences classiques. Cette différence de coût peut influencer les choix des consommateurs et encourager l'adoption de carburants alternatifs lorsque les véhicules le permettent.

Analyse du nombre de stations distribuant chaque type de carburant en France :



Ce graphique présente le **nombre de stations distribuant chaque type de carburant en France**.

On voit que :

- le **Gazole** est le carburant le plus largement distribué, disponible dans environ **9 600 stations**.
- le **SP98** est présent dans environ **8 200 stations**.
- l'**E10** est disponible dans près de **7 700 stations**.
- l'**E85** est proposé par environ **4 000 stations**.
- le **SP95** est disponible dans environ **3 600 stations**.
- le **GPLc** est le carburant le moins répandu, avec seulement **1 500 stations** environ.

Le **Gazole** reste le carburant le plus accessible. Malgré la baisse progressive de sa part dans les nouvelles immatriculations, il demeure le carburant le plus distribué sur le territoire. Cela reflète l'importance historique du parc automobile diesel en France. Les **essences** sont très largement disponibles. Le **SP98** et l'**E10** bénéficient d'un réseau de distribution très étendu. Cette forte présence facilite leur accès pour les automobilistes et explique en partie leur forte consommation. Les **carburants alternatifs** disposent d'un réseau

plus limité L'**E85** est présent dans moins de la moitié des stations distribuant du **Gazole**. Le **GPLc** reste le carburant le moins accessible, ce qui peut freiner son adoption malgré son coût relativement faible.

L'analyse met en évidence une forte disparité dans la couverture du réseau de distribution selon les carburants. Le **Gazole**, le **SP98** et l'**E10** dominent largement le marché en termes de disponibilité, tandis que l'**E85** et surtout le **GPLc** restent des carburants plus spécialisés. La combinaison de ces résultats avec les données de prix et de ruptures permet de mieux comprendre les habitudes de consommation et les enjeux logistiques du marché des carburants en France.

Les captures d'écran suivantes présentent des exemples de visualisations interactives : il est possible de sélectionner le carburant et la ville de son choix grâce aux filtres disponibles. Les graphiques se mettent alors automatiquement à jour afin d'afficher les données correspondant aux critères sélectionnés.

Evolution du prix moyen dans le temps par carburant :



Ce graphique montre l'**évolution du prix moyen du Gazole à Paris sur la dernière année**.

On voit :

- qu'**entre juin et août 2025**, le prix du **Gazole** varie entre **1,60 €** et **1,68 €/L**, avec quelques variations modérées.
- que de **septembre à octobre 2025**, les prix restent relativement stables autour de **1,62 €** à **1,65 €/L**.
- qu'une hausse marquée est observée à partir de **novembre 2025**, atteignant un pic proche de **1,73 €/L** vers la fin du mois.

- qu'après ce maximum, le prix diminue progressivement en **décembre** pour revenir autour de **1,59 €/L**, niveau proche de celui observé en début de période.

D'après le graphique :

- Prix minimum : $\approx 1,59$ €/L
- Prix maximum : $\approx 1,73$ €/L
- Amplitude : $\approx 0,14$ €/L
- Prix moyen estimé : $\approx 1,64$ €/L

Le prix du Gazole à Paris est resté globalement stable sur l'année, avec une hausse notable à l'automne suivie d'un retour rapide à des niveaux plus habituels. Les variations observées restent relativement limitées, ce qui indique que le marché du Gazole a connu peu de perturbations durables sur la période étudiée. Cette analyse montre également l'intérêt d'un suivi temporel pour identifier les périodes de hausse ou de baisse et mieux comprendre l'évolution des prix des carburants.

Comparaison des stations par ville et par carburant :

Choisir un carburant

Gazole

Choisi la ville

PARIS

Table view

station	prix
2/10 BLD FORT DE VAUX (PARIS)	1.59 €
85 RUE JEANNE DARC (PARIS)	1.71 €
234 RUE DE CRIMEE (PARIS)	1.73 €
2 AV DE LA PORTE DE ST CLOUD (PARIS)	1.73 €
217 BOULEVARD MARECHAL DAVOUT (PARIS)	1.73 €
164 Boulevard Haussmann (PARIS)	1.73 €
30 AV PTE CLIGNANCOURT (PARIS)	1.73 €
32 AVENUE DE LA PORTE DE ST OUEN (PARIS)	1.73 €
121 RUE MANIN (PARIS)	1.74 €

Ce tableau présente une **comparaison des stations-service de la ville de Paris pour le carburant Gazole**.

Chaque ligne correspond à une station et affiche le prix pratiqué pour ce carburant. On observe que les prix sont relativement proches d'une station à l'autre, variant d'environ **1,59 € à 1,74 €/L**. La station située au 2/10 Boulevard Fort de Vaux propose le prix le plus bas observé dans cet extrait (1,59 €/L), tandis que plusieurs stations affichent des prix compris entre 1,73 € et 1,74 €/L.

Cette faible dispersion des prix suggère une certaine homogénéité du marché du Gazole à Paris, même si quelques écarts permettent aux consommateurs de réaliser des économies en choisissant les stations les moins chères.

Grâce aux filtres du tableau de bord, il est possible de sélectionner n'importe quelle ville ainsi que n'importe quel carburant. Le tableau se met alors automatiquement à jour pour afficher les stations correspondantes et leurs prix, facilitant ainsi la comparaison entre différentes zones géographiques et différents types de carburants.

Top 10 des stations les moins chères par carburants :

Choisir un carburant	Gazole ▾	Choisi la ville	PARIS ▾	Table view ☐
Top 10 des stations les moins chères par carburants				⋮
ville	adresse	prix_fixe	date_maj	
FOIX	Route d'Espagne	1.21 €	2025-04-18 09:38:34	
SAINT-NAZAIRE	54 Rue Léon Bourgeois	1.21 €	2025-12-22 06:12:56	
Vernou-sur-Brenne	25 Rue du Professeur Robe	1.22 €	2025-06-06 09:22:28	
BOISSISE-LA-BERTRAND	Avenue Jean Monnet	1.22 €	2025-04-18 11:19:20	
Auxonne	AVENUE GENERAL DE GAU	1.23 €	2025-05-05 14:10:00	
Oyonnax	74 Rue Jules Michelet	1.23 €	2025-12-16 09:03:48	
Avignon	10 Avenue Eisenhower	1.23 €	2025-12-12 09:28:06	
Erquinghem-Lys	Rue d'Armentières	1.23 €	2025-04-19 12:16:18	
BOISSISE-LA-BERTRAND	Avenue Jean Monnet	1.24 €	2025-04-18 11:12:46	
AVORD	2 rue Agnès Sorel	1.24 €	2025-04-18 12:07:50	

Ce tableau présente le **Top 10 des stations-service proposant le Gazole au prix le plus bas** parmi les données disponibles.

Pour chaque station, sont indiqués la ville, l'adresse, le prix du carburant ainsi que la date de la dernière mise à jour du tarif. On constate que les prix les

plus faibles se situent entre 1,21 € et 1,24 € par litre, ce qui est nettement inférieur au prix moyen observé précédemment.

Les stations les moins chères sont réparties dans plusieurs villes françaises telles que Foix, Saint-Nazaire, Avignon, Oyonnax ou Auxonne, ce qui montre que les prix les plus avantageux ne sont pas concentrés dans une seule région.

Les écarts de prix entre les stations du classement restent relativement faibles, mais ils peuvent représenter une économie significative pour les automobilistes effectuant des pleins réguliers.

Cette visualisation permet ainsi d'identifier rapidement les stations les plus compétitives et de comparer les tarifs pratiqués sur le territoire.

Grâce aux filtres du tableau de bord, l'utilisateur peut sélectionner le carburant de son choix ainsi qu'une ville spécifique. Le classement est alors automatiquement recalculé afin d'afficher les stations les moins chères correspondant aux critères sélectionnés, offrant une analyse personnalisée et adaptée aux besoins de chacun.

4.2. Synthèse des résultats obtenus

L'objectif principal de cette SAÉ était de construire une base de données relationnelle capable de stocker, d'historiser et d'exploiter des données ouvertes relatives aux stations-service françaises. À l'issue de cela, nous avons obtenu divers résultats qui justifient pleinement que l'objectif a été atteint.

Tout d'abord, l'analyse des premières informations nous a permis de construire un modèle relationnel représentant pleinement la situation avec six tables : STATION, CARBURANT, PRIX, SERVICE, RUPTURE et PROPOSE. Grâce à cela, nous avons limité les redondances et respecté les différentes contraintes liées aux attributs. Par exemple, un carburant est obligé d'avoir un nom. De plus, le script Python développé a permis d'extraire automatiquement les informations du fichier XML national des prix des carburants.

En effet, en une lecture du fichier nous avons obtenu différents fichiers CSV représentant les carburants, les services, les ruptures et les horaires d'ouverture. Par la suite, nous avons fait l'usage de tables temporaires afin de faciliter le chargement des CSV et de faciliter la transformation des données avant leur importation. Cela permet d'éviter les doublons et de vérifier les données importées.

Nous avons également réalisé des requêtes SQL afin d'exploiter les données de notre base. Les résultats obtenus montrent que les jointures entre les tables fonctionnent correctement et donnent des résultats proportionnels à la réalité. En effet, l'analyse des prix montre un écart entre les stations. Par exemple, on a pu voir lesquelles des stations étaient les plus avantageuses vis-à-vis du prix.

On a aussi pu constater via la requête des ruptures de stock que certains carburants étaient plus souvent amenés à être en rupture, comme le SP-98. L'analyse des horaires permet aussi de répartir les stations entre celles ouvertes 24h/24h et celles avec des horaires plus restreints, montrant ainsi le niveau d'automatisation des stations.

Les différents prix observés sur les différents carburants correspondent bien aux tendances actuelles, avec le SP-98 qui apparaît comme l'un des plus chers sur le marché.

Enfin, nous avons établi un tableau de bord avec Grafana afin de rendre les résultats de nos requêtes plus faciles à visualiser.

Le graphique représentant le nombre de ruptures par carburant met en évidence les carburants les plus concernés par les problèmes d'approvisionnement. Le tableau des dix stations les moins chères permet quant à lui d'identifier rapidement les meilleures offres disponibles pour chaque carburant.

On peut suivre la variation des prix du carburant grâce à l'analyse temporelle du prix moyen.

En conclusion, grâce à ces visualisations, on peut prendre des décisions rapidement et fondées sur des informations actualisées.

4.3. Conclusion sur les principaux enseignements du projet

Cette SAÉ nous a permis de mettre en pratique l'ensemble des étapes nécessaires à l'exploitation d'une base de données, depuis la modélisation jusqu'à la visualisation des résultats. À partir de données ouvertes sur les stations-service françaises, nous avons conçu une base de données relationnelle capable de stocker, organiser et historiser efficacement les informations relatives aux carburants, aux prix, aux services et aux ruptures de stock.

La mise en place du tableau de bord Grafana a permis de rendre ces analyses plus accessibles grâce à des visualisations interactives. Les filtres par ville et par carburant offrent la possibilité d'explorer les données selon différents critères et d'obtenir rapidement des informations pertinentes pour comparer les prix, suivre leur évolution ou identifier les stations les plus avantageuses.

Enfin, ce projet nous a permis de développer des compétences en modélisation de données, en SQL, en traitement de données avec Python et en visualisation avec Grafana. Il illustre l'importance des bases de données dans l'exploitation de grands volumes de données et dans l'aide à la prise de décision à partir d'informations fiables et actualisées.

[#Retour au début](#)